

第七届全国校园人工智能算法精英大赛-算法应用赛
智慧社区 省赛技术报告模板

一、 基本信息

队伍名称	超越者
参赛编号	AIC-2025-45861355
参赛队员	孙郝蓬、金子越、毕智铖
摘要	本文围绕赛题要求，设计并实现了一套基于ROS的智慧社区安防机器人仿真系统。在Gazebo仿真环境中搭建了包含道路、墙壁、标识立牌等元素的社区场景，并利用XACRO构建了具备多传感器的轮式机器人模型。通过自主设计的路径规划与图像采集模块，实现了机器人在预设路径上的精确导航与多目标点图像采集。在视觉识别方面，系统集成基于YOLOv8的人员身份分类模型与基于PP-OCRv3的车牌识别模型，结合图像增强与几何校正技术，有效提升了复杂环境下的识别准确率。测试结果表明，系统在导航精度、图像采集与分类、人员与车牌识别等方面均达到预期目标，具备良好的稳定性与实用性。本研究为智慧社区治理提供了可落地的技术验证方案，展示了人工智能在社区安防中的广泛应用前景。 关键词： 智慧社区；ROS；路径规划；YOLOv8；PP-OCRv3；图像识别；仿真系统

1. 引言

1.1 赛题理解与背景分析

智慧社区建设作为城市现代化治理的重要环节，正面临着传统人力管理模式在效率、成本和响应速度上的多重挑战。随着城镇化进程加速和人口流动性增强，社区在人员出入管控、安防巡逻、车辆停放管理等典型场景的管理需求日益复杂，亟需通过技术手段提升治理效能。本次竞赛以社区治理实际需求为切入点，要求参赛者构建具备环境感知与自主决策能力的智能机器人系统，正是顺应了人工智能技术与城市管理深度融合的发展趋势。竞赛任务包含仿真环境搭建、运动控制与视觉识别三大技术模块，需要在ROS框架下完成从物理建模到算法部署的全流程开发，既考察了对三维场景的精确还原能力，也检验了路径规划与多目标视觉识别的技术整合水平。通过构建数字孪生环境并实现智能巡检功能，为探索低成本、高效率的社区治理解决方案提供了技术验证平台方案。

1.2 核心挑战与解决思路

在虚拟化环境平台性能支撑方面，由于竞赛开发工作完全在虚拟机环境中开展，在Gazebo上搭建仿真环境及令其稳定运行，成为我们初期面临的首要技术障碍。这要求我们不仅要从小虚拟机本身资源配置角度进行优化调整，更需要从仿真参数设置和系统运行策略层面设计针对性方案，通过综合优化来弥补虚拟化环境固有的性能缺陷，确保整个系统在有限硬件资源条件下仍能保持必要的运行效率和稳定性，为后续各项功能的开发验证提供可靠的基础环境支撑。

在基础模型功能扩展与定制化改造方面，面对官方提供的基础模型结构与功能相对简单原始的客观现实，如何基于这些基础模型进行必要的功能增强和性能优化，使其能够满足竞赛任务提出的各项复杂功能需求，构成了项目推进过程中的又一重要技术课题。这要求我们深入理解模型文件的结构组成与接口规范，通过合理的结构修改与参数调整，在保持模型兼

容性的前提下实现功能的灵活扩展,这种在约束条件下进行模型优化的过程实际上是对我们技术理解深度和工程实现能力的综合考验。

在机器人运动控制精度保障这一核心技术环节,针对轮式机器人在固定路径行驶过程中难以避免的轨迹偏移与定位误差问题,我们需要在不过度依赖高精度传感器且不引入复杂控制算法的前提条件下,设计实现一套简洁有效的轨迹校正机制。这种机制既要保证机器人能够准确遵循预定路径完成行进任务,又要确保方案不会对系统资源造成过大压力。

在视觉识别模型训练与优化方面,受限于训练样本数量有限、类别单一的客观条件,如何利用有限的标注数据训练出具有足够泛化能力的识别模型,成为影响任务完成质量的关键因素之一。这要求我们在模型架构选择、数据增强策略和训练方法优化等多个环节进行精心设计和反复调试,通过技术手段弥补数据资源不足的短板,这种在小样本条件下的模型优化过程对我们的技术创新能力和工程实践经验都提出了较高要求。

2. 系统总体设计

2.1 设计目标

本系统的设计目标旨在构建一个集环境感知、自主决策与智能识别于一体的智慧社区安防机器人系统。通过深度融合机器人导航技术与计算机视觉算法，拟实现从仿真环境搭建到实际功能落地的全流程技术闭环。

在自主导航方面，系统拟设计基于坐标点驱动算法的多目标点路径跟踪机制，使机器人能够沿预定路径精确移动，并在指定识别区域实现稳定停靠。

在视觉感知层面，系统拟建立基于位置触发的图像采集与处理流水线。当机器人到达关键路径点时，将自动激活图像采集程序，并根据场景语义将数据分类存储至相应目录。

针对社区安防需求，系统拟开发双模态视觉识别能力。人员识别模块计划基于YOLOv8架构，实现对社区人员与非社区人员的准确区分；车牌识别模块拟采用PP-OCRv3框架，通过文本检测、方向校正与字符识别的多阶段处理，确保复杂环境下的识别可靠性。

2.2 系统架构图与技术路线

2.2.1 系统架构

系统架构图如图2-1。

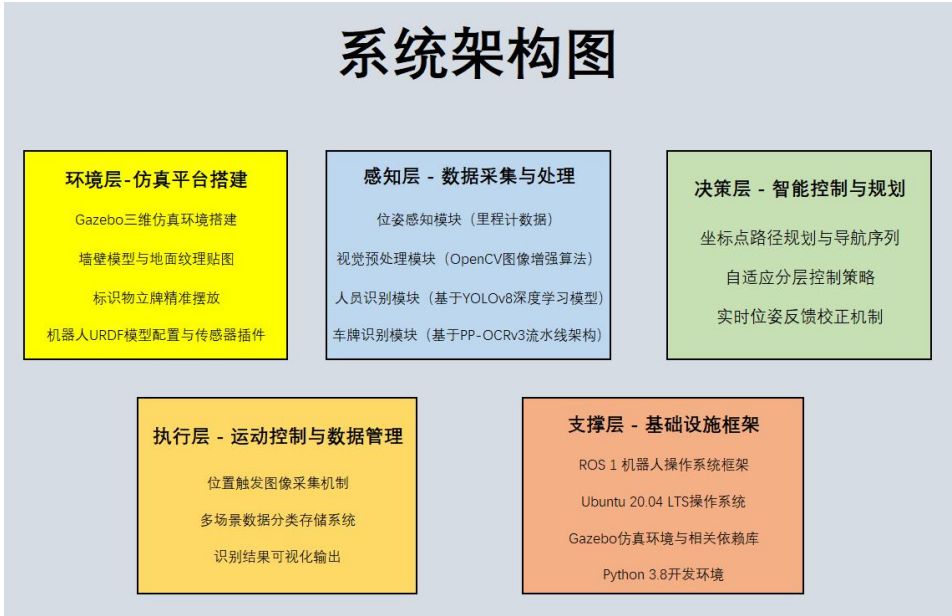


图2-1 系统架构图

2.2.2 技术路线

在第一阶段中，我们首先完成了仿真环境的搭建工作。具体来说，利用Gazebo Building Editor创建了墙壁模型，并导入地面纹理贴图，从而建立起所需的仿真地图。随后，按照要求将三个标识物立牌摆放到场景中。在机器人模型开发方面，我们使用XACRO宏语言定义了机器人的URDF模型，同时配置了差速驱动系统和传感器，包括摄像头与激光雷达，并通过集成Gazebo插件实现了物理仿真功能。

进入第二阶段后，我们集中开发了核心算法。其中，自主导航系统基于PID控制实现了多目标点的路径跟踪，同时结合实时位姿感知与反馈控制，进一步引入了自适应速度调节与方向校正机制，以提升导航的精确性。在视觉识别流水线部分，我们针对人员识别采用了基于YOLOv8的身份分类模型，而在车牌识别方面则应用了基于PP-OCRv3的端到端识别方案。此外，图像预处理环节还加入了对比度增强与色彩空间转换，以优化识别效果。

第三阶段主要围绕系统集成测试展开。我们首先构建了ROS节点网络并实现了话题通信，确保各功能模块能够协同工作，同时加强了数据流管理，并设计了异常处理与系统容错机制。随后，在性能优化验证过程中，对导航精度进行了测试与调优，评估并改进了识别准确率，最终验证了系统的整体稳定性与实时性表现。

3. 核心技术与模块实现

3.1 仿真平台搭建

3.1.1 静态模型建立

仿照gazebo自带的模型ground_plane建立了新模型runway。插入比赛要求的“地图”图片，以其为模型的纹理，并调整了材质的相关属性，使之清晰可见，如图3-1。再通过gazebo的building editor 功能建造了模型walls， 4.2×4.2 的四面墙，并设置了合适的厚度与高度，材质使用的默认的Gazebo/Grey。

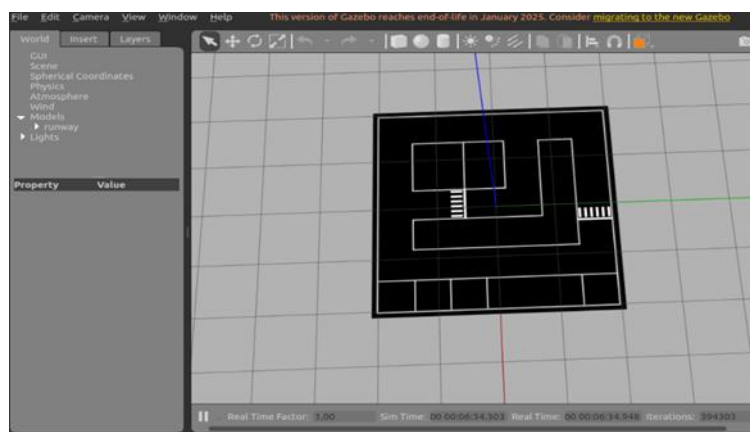


图 3-1 道路模型

2、立牌模型

根据比赛要求插入所给模型，更改3个模型中的变换矩阵（matrix sid="transform"）是立牌朝向精确，再将其放入gazebo的模型库如图3-2。

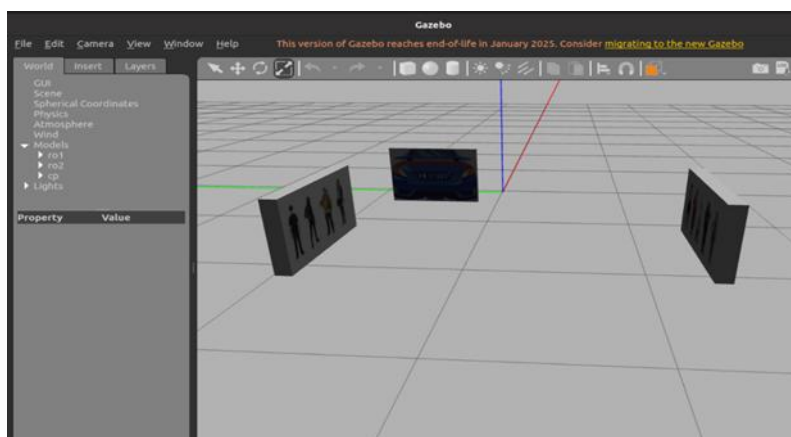


图 3-2 立牌模型

3. 建立仿真地图

在gazebo中插入模型立牌（ros1、ros2、cp），地图（runway，walls），光源（sun）

如图3-3。

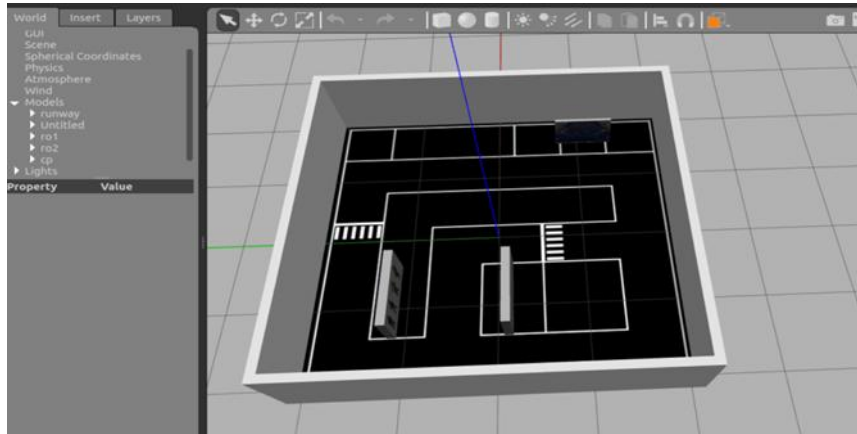


图 3-3 完整地图

最后将完整的仿真地图保存为runway3.world文件。

3.1.2 动态模型建立

1、创建完整的机器人模型描述文件

使用XACRO这一URDF的宏语言来实现模块化、参数化的模型设计，定义基础惯性矩阵的惯性参数计算，然后通过编写多个.xacro格式的文件来构建完整的机器人模型。这些文件采用主文件集成子组件的架构模式，通过xacro:include指令将底盘、车轮、传感器等子模块有机整合，其中底盘模块定义了基础链接的几何属性和物理特性，驱动轮和支撑轮通过宏定义实现参数化生成，摄像头和雷达模块则分别配置了相应的安装位置和传感特性。

2、仿真环境的集成启动与机器人控制

通过Launch文件实现一站式部署。这个启动文件需要完成多个关键任务：首先将XACRO文件解析为URDF格式并加载到参数服务器，接着启动指定的Gazebo仿真环境，然后通过spawn_model节点将机器人模型实例化到仿真世界中。为了支持机器人导航功能，需要配置并启动必要的传感器插件，包括：

- （1）差分驱动控制器：接收`/cmd\_vel`控制指令并驱动机器人底盘运动

- (2) 激光雷达传感器：提供环境扫描数据
- (3) 摄像头传感器：发布`/camera/image\_raw`图像话题用于视觉任务
- (4) IMU传感器：提供惯性测量数据
- (5) 里程计系统：发布`/odom`话题提供机器人实时位姿信息

同时，系统会启动robot_state_publisher节点来维护机器人的坐标变换体系，确保TF树的完整性。完成这些准备工作后，通过roslaunch命令执行启动文件即可在Gazebo中看到完整的机器人仿真环境，此时开发者可以通过向`/cmd\_vel`控制话题发布`geometry\_msgs/Twist`类型的速度指令，实现对机器人的精确运动控制。机器人会通过里程计系统实时反馈位置信息，并通过摄像头传感器采集环境图像，从而完成从模型构建到感知控制的完整仿真流程。

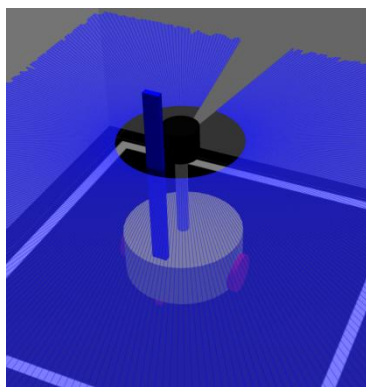


图 3-4 仿真机器人小车

3.2 软件核心框架与开发环境

本项目基于ROS1框架，在Ubuntu 20.04系统上，主要采用Python3.8进行开发。系统利用YOLOv8目标检测模型与OpenCV库构建核心视觉感知模块，并利用 PIL 、 numpy 等关键库实现图像处理与数值计算。

功能包在本项目下命名为“ceshi_g”，其内部文件架构如图3-5。

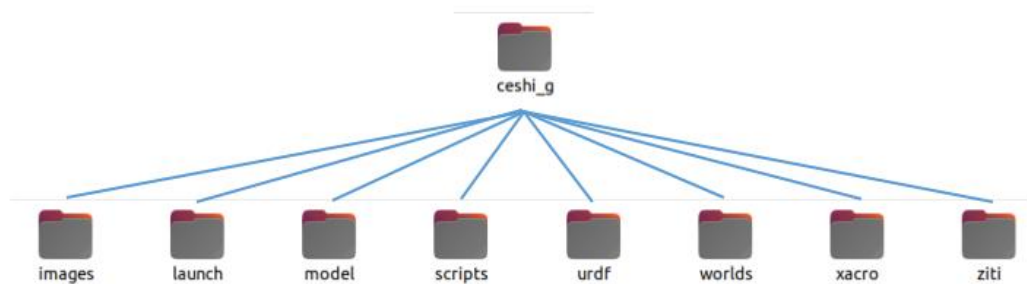
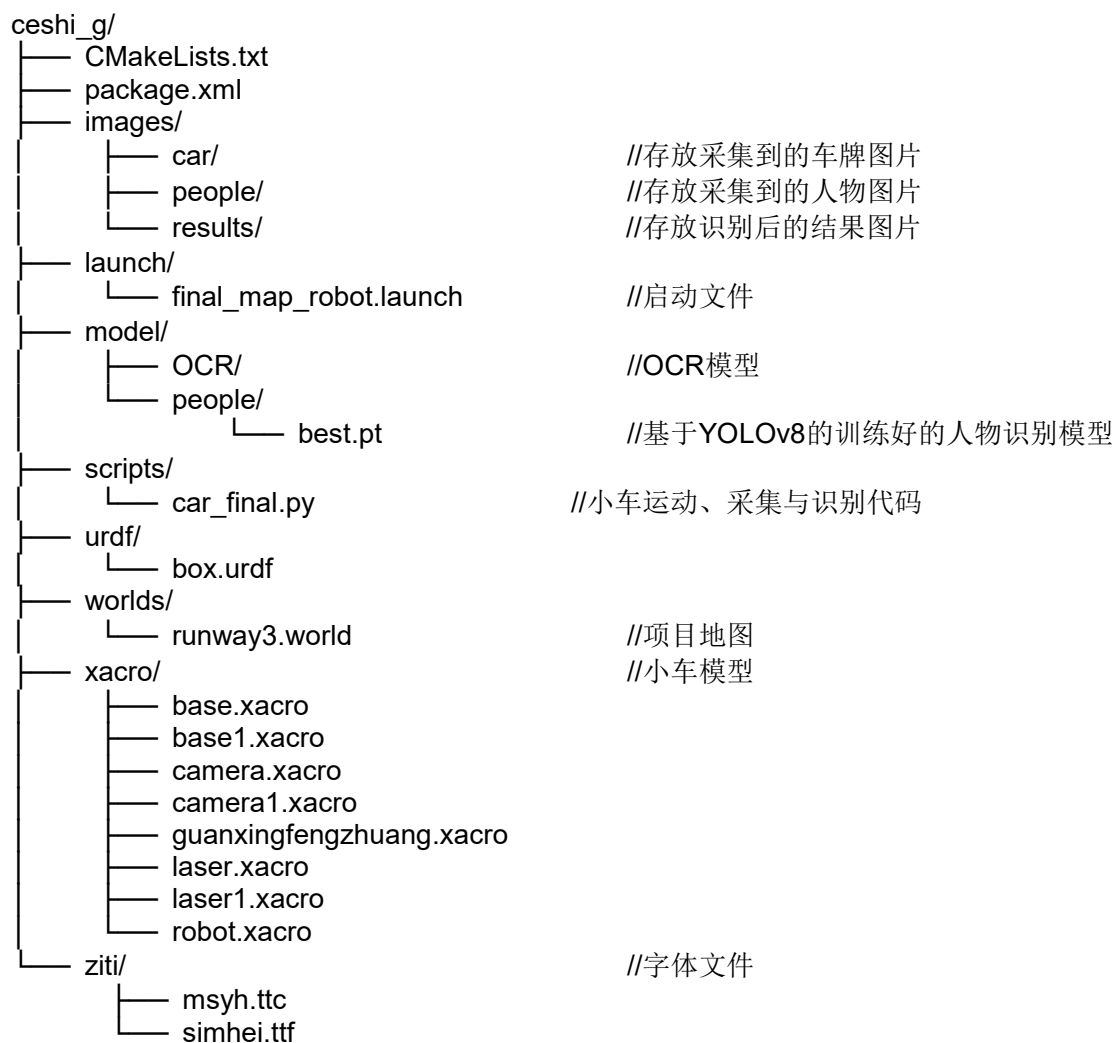


图3-5 “ceshi_g” 文件架构

更为详细的文件架构如下：



3.3 关键模块一：路径规划与图像采集模块

3.3.1 功能描述：

本模块设计的机器人控制系统是一个基于ROS框架的移动机器人导航与图像采集系统。

该系统具备以下核心功能：

1、多目标点自主导航功能：系统能够控制移动机器人按照预设的路径点序列进行精确导航。路径点序列包含9个关键位置坐标，机器人需依次访问每个目标点，并在特定位置执行额外的方向调整和图像采集任务。

2、实时位姿感知与反馈控制：系统通过订阅里程计Odometry话题实时获取机器人的当前位置和朝向信息。基于这些实时数据，系统能够计算机器人与目标点之间的相对位置和方向偏差，并生成相应的控制指令。

3、自适应运动控制：系统采用分层控制策略，根据机器人与目标点的相对位置关系动态调整运动模式。当方向偏差较大时，优先进行原地旋转对准；当方向基本对准后，执行直线前进运动，同时在运动中保持小角度修正。

4、多场景图像采集与分类存储：系统集成了视觉感知能力，通过摄像头模块在特定位置采集环境图像。根据场景语义的不同，系统能够自动将采集的图像分类存储：人物场景图像保存至“people”目录，车辆场景图像保存至“car”目录。每个图像文件均包含位置、朝向和时间戳等元数据信息。

5、精确方向控制与保持：在指定的采集点，系统能够控制机器人旋转至精确的目标方向：正北 90° 、正南 -90° 、正东 0° ，并保持该方向稳定一段时间，确保采集图像的清晰度和一致性。

3.3.2 算法原理/实现方法：

本模块采用基于位置伺服的双层控制架构，结合条件触发的数据采集机制，实现精确导航与智能感知的协同工作。

路径规划算法实时计算机器人当前位置与目标点的欧几里得距离和方向角度偏差，建立误差动力学模型。距离控制采用线性比例调节器，根据剩余距离动态调整前进速度，在接近

目标时自动减速以确保定位精度。角度控制采用分段策略：当方向偏差大于阈值 8.6° 时，优先执行纯旋转运动；当方向偏差较小时，结合前进运动进行细微方向修正。

方向控制算法基于角度误差归一化处理，将方向偏差映射至 $[-\pi, \pi]$ 区间，避免角度跨越 2π 边界时出现控制震荡。采用比例控制律生成角速度指令，通过增益调度机制实现快速对准与精确稳定的平衡。在达到目标方向后，启动方向保持模式，通过闭环反馈维持指定朝向，为图像采集提供稳定的观测平台。

图像采集机制采用事件触发模式，在机器人到达特定坐标点并完成方向对准后自动激活。采集系统首先等待传感器数据稳定，确保图像质量；然后根据位置上下文信息自动分类图像类型，生成包含时间戳、位置信息和朝向角度的标准化文件名；最后将图像数据存储至预设的目录结构中。

性能监控系统实时记录各导航阶段的运动参数，包括实际行进距离、位置误差、方向精度和任务完成时间等关键指标。通过数据统计分析，为系统优化提供量化依据，确保导航精度与采集质量的持续提升。

3.3.3 难点与对策：

1、导航精度与稳定性平衡难题：机器人在接近目标点时，由于惯性作用容易产生位置超调现象，同时在方向快速调整时可能引起机身晃动，影响图像采集质量。对此，本系统采用自适应速度调节策略，基于剩余距离线性降低前进速度，实现平滑减速。

2、多传感器数据同步问题：里程计数据与图像采集之间存在时间延迟，可能导致位置记录与实际情况不匹配。系统通过数据时间戳对齐技术和状态等待机制，确保在机器人运动稳定后再触发图像采集，避免运动模糊对图像质量的影响。

3.4 关键模块二：视觉识别模块1（识别是否社区人员）

3.4.1 功能描述：

1、图像预处理与增强功能：系统首先对原始采集图像进行预处理操作，包括图像尺寸调整和对比度增强。通过参数化调节机制，能够有效改善图像质量，提升后续目标检测的准确率。增强处理采用平衡化策略，在保持图像自然度的同时显著提升特征可辨识度。

2、人员检测与定位功能：模块能够自动识别图像中所有人员目标，精确标定其位置信息，并输出每个检测目标在图像中的相对坐标、宽度和高度参数，建立完整的目标空间分布图谱。

3、身份分类识别功能：在检测到人员目标的基础上，模块进一步实现身份分类。系统将检测到的人员区分为“社区人员”与“非社区人员”两个类别。

4、批量处理与统计分析功能：模块支持对图像文件的批量自动化处理，能够依次处理指定目录下的所有图像文件。系统自动统计每张图像的识别结果，并输出总人数、分类人数和位置信息。

5、可视化结果输出功能：在完成识别任务后，模块自动保存带有检测框和类别标签的结果图像。通过在原文件名基础上添加“_result”后缀，建立原始数据与处理结果的对应关系，便于结果验证与分析。

3.4.2 算法原理/实现方法：

本模块基于YOLOv8目标检测架构，采用深度学习实现人员识别任务。

1、基于对比度拉伸的图像增强算法：观察到采集到的图像较暗，系统首先采用线性变换方法进行图像增强，具体公式为：

$$g(x) = \alpha \cdot f(x) + \beta$$

其中， $f(x)$ 为原始像素值， $g(x)$ 为增强后像素值。通过精心调整对比度增益参数 α （本代码取1.1）和亮度偏移参数 β （本代码取25），实现在增强图像细节的同时避免过度饱和。这种平衡增强策略特别适用于人员检测任务，能够有效突出人物特征同时保持背景信息的完整性。

2、基于YOLOv8深度学习的模型训练：采用YOLOv8n预训练模型作为基础网络，通过迁移学习技术，在自定义的人员识别数据集上进行微调训练，训练周期设置为200个epoch，批处理大小为8，确保模型充分收敛。训练好的新模型命名为“best.pt”。

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size				
12/200	1.156	0.8689	0.9961	1.04	46	640: 100%		8/8	[00:00<00:00, 12.76it/s]	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		1/1	[00:00<00:00, 21.83it/s]
	all	9	35	0.861	0.944	0.966	0.72			
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size				
13/200	1.146	0.8992	0.9918	1.055	53	640: 100%		8/8	[00:00<00:00, 12.65it/s]	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		1/1	[00:00<00:00, 20.74it/s]
	all	9	35	0.774	0.904	0.935	0.525			
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size				
14/200	1.156	0.8472	0.9644	1.02	40	640: 100%		8/8	[00:00<00:00, 12.75it/s]	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		1/1	[00:00<00:00, 22.49it/s]
	all	9	35	0.831	0.788	0.914	0.611			
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size				
15/200	1.156	0.8428	0.9093	1.019	42	640: 100%		8/8	[00:00<00:00, 12.73it/s]	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		1/1	[00:00<00:00, 22.94it/s]
	all	9	35	0.809	0.889	0.886	0.674			
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size				
16/200	1.156	0.8434	0.8651	0.9898	66	640: 100%		8/8	[00:00<00:00, 12.87it/s]	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		1/1	[00:00<00:00, 22.92it/s]
	all	9	35	0.667	0.77	0.753	0.531			
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size				
17/200	1.156	0.8509	0.8499	0.9991	55	640: 100%		8/8	[00:00<00:00, 11.79it/s]	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		1/1	[00:00<00:00, 22.00it/s]
	all	9	35	0.861	0.793	0.855	0.604			

图3-6 模型训练过程1



图3-6 模型训练过程2

3、推理检测机制：模型对输入图像进行前向传播，生成包含边界框坐标、类别置信度和类别标签的检测结果。采用非极大值抑制算法消除重叠检测框，确保每个目标只被检测一次。关键输出包括目标中心坐标(x_center , y_center)、宽度(width)、高度(height)等归一化几何参数。

4、结果后处理与统计：检测结果按类别进行聚合分析，分别统计“社区人员”（类别0）和“非社区人员”（类别1）的数量和空间分布。系统为每个检测目标生成详细的位置描述，包括相对坐标信息和边界框尺寸。

3.4.3 难点与对策：

1、图像质量不一致的挑战：原始采集图像受光照条件、角度、距离等因素影响，质量有着提高空间。本系统采用图像增强技术，通过 α 、 β 参数组合确保处理一致性。

2、尺度变化适应：人员目标在图像中呈现多尺度特性，远近大小差异显著。YOLO模型含有多尺度训练策略和特征金字塔网络结构，增强模型对不同尺度目标的检测能力。

3、ultralytics库占用内存较大，ubuntu内存空间不足。将内存空间增大并分配即可。

3.5 关键模块三：视觉识别模块2（识别车牌号）

3.5.1 功能描述：

车牌号识别模块实现了从复杂背景中准确提取和识别车牌信息的关键功能。该模块具备以下核心能力：

1、图像预处理与增强功能：系统首先对原始采集的车辆图像进行预处理，包括图像尺寸调整和对比度增强，结合LAB色彩空间转换、CLAHE对比度限制直方图均衡化和锐化滤波等技术，有效改善图像质量，消除光照不均和分辨率差异对后续识别的影响。

2、车牌区域精确定位功能：系统基于预设的车牌坐标信息，结合图像缩放比例自动调整定位区域，确保在不同尺寸图像中都能准确锁定车牌位置。通过边界矩形计算和透视变换校正，消除拍摄角度造成的几何畸变，为后续识别提供标准化的输入数据。

3、端到端字符识别功能：基于PP-OCRv3流水线架构，模块实现了从车牌图像到文本信息的端到端识别。系统依次执行文本检测、方向分类和字符识别三个子任务，确保对中文、英文字母和数字的混合文本具备准确识别能力。

4、结果后处理与验证功能：模块内置智能文本清理机制，通过正则表达式过滤和车牌格式验证，去除识别结果中的噪声字符。系统依据中国车牌编码规则对识别结果进行合理性校验，输出符合规范的车牌号码。

3.5.2 算法原理/实现方法：

本模块采用基于PP-OCRv3的三阶段流水线识别框架，结合传统图像处理与深度学习技术。

1、基于线性变换的图像增强算法：使用与3.4.2中第一点相同的图像增强算法。随后针对车牌区域应用LAB色彩空间转换与CLAHE局部对比度增强，最后通过锐化滤波强化字符边缘特征。

2、坐标自适应调整机制：系统建立了一套完整的坐标映射体系，根据图像缩放比例自动调整车牌定位坐标。通过scale_factor参数实现原始坐标到增强图像坐标的精确转换，确保在不同分辨率下都能准确定位车牌区域。

3、多模块协同的OCR识别架构：

（1）文本检测阶段采用DB（Differentiable Binarization）文本检测算法，通过可微分二值化过程实现文本区域的精确分割。检测器参数经过针对性优化，设置检测阈值0.3、框阈值0.6和非闭合比率1.5，平衡召回率与精确度的需求。

（2）方向分类阶段基于轻量级分类网络，对检测到的文本区域进行方向判别。当分类置信度超过0.9阈值时，系统自动对倾斜文本进行180度旋转校正，确保字符处于标准阅读方向。

（3）字符识别阶段采用CRNN（Convolutional Recurrent Neural Network）架构结合注意力机制，实现序列到序列的字符识别。识别模型在包含中文字符、英文字母和数字的专用词典上进行训练，支持车牌特有的字符集组合。

4、基于透视变换的几何校正：针对车牌图像可能存在的视角畸变，系统通过四对对应点的坐标映射计算透视变换矩阵，实现倾斜车牌的正面化校正，为后续OCR识别提供标准化的输入图像。

5、多层级结果验证机制：系统构建了从字符级到文本级的多层级验证体系。字符级验证通过预定义白名单过滤无效字符，文本级验证基于车牌长度规范进行合理性判断，确保输出结果的可靠性。

3.5.3 难点与对策：

1、背景干扰问题：车辆图像中包含大量非车牌区域的纹理和边缘信息。对策是通过精确的坐标定位和区域提取，隔离车牌区域；结合透视变换消除透视畸变，聚焦核心识别目标。

2、复杂光照条件下的字符识别挑战：光照不均、反光等问题会导致字符特征模糊。应用LAB色彩空间增强技术，在保持颜色真实性的同时显著提升对比度；结合CLAHE算法自适应调整局部对比度，确保在不同光照条件下都能提取清晰字符特征。

3、字符分割粘连难题：车牌字符间可能存在粘连或断裂现象。DB文本检测器的可微分二值化处理能够有效处理字符间距变化；CRNN序列识别架构避免显式字符分割，直接从图像特征映射到文本序列。

4、车牌透视畸变影响识别精度：拍摄角度导致的几何变形会严重影响字符识别效果。采用透视变换校正技术，通过四对对应点的坐标映射，将倾斜车牌校正为标准矩形，消除视角引起的几何畸变。

5、中文显示的技术实现难题：OpenCV原生不支持中文字符渲染。结合PIL库的图像处理能力，实现中文字符的绘制功能，并建立多字体路径备选机制，确保在不同系统环境中都能正常显示中文识别结果。

3.6 本项目创新点

1、多模态感知与导航的深度集成：本项目创新性地实现了机器人导航与视觉感知的深度融合，突破了传统机器人系统中感知与决策模块相互独立的局限。通过建立基于位置触发

的图像采集机制，系统能够在预设的关键路径点自动执行多角度图像采集任务，并将采集数据按场景类型智能分类存储。这种集成架构不仅提升了数据采集的针对性和效率，更为后续的智能分析提供了高质量、结构化的数据基础。相较于传统的随机采集方式，本系统的定向采集策略显著提高了数据的有效性和利用率。

2、自适应分层控制策略：在运动控制方面，本项目提出了一种基于双阶段策略的自适应分层控制方法。该系统根据机器人与目标点的相对位置和方向偏差，动态调整运动模式：在远距离时采用快速接近策略，在近距离时切换为精确定位模式。特别地，系统引入了角度误差归一化处理和死区补偿机制，有效解决了传统PID控制在角度跨越 2π 边界时出现的震荡问题。这种分层控制策略在保证导航精度的同时，显著提升了系统的稳定性和响应速度。

3、系统级的可靠性与实用性设计：本项目在系统架构层面进行了多项创新性设计，以确保在真实社区环境中的长期稳定运行。通过设计多传感器数据同步机制，系统具备了较强的容错能力。同时，系统提供了完整的可视化输出和统计分析功能，支持操作人员对系统状态和识别结果进行直观监控和评估。这些设计使得系统不仅具备先进的技术性能，更具有良好的实用性。

4、端到端的智能化解决方案：本项目最重要的创新在于提供了一个完整的端到端智能社区安防解决方案。从自主导航、数据采集到智能识别和结果分析，系统实现了全流程的自动化处理。通过将机器人技术、计算机视觉技术和智能控制技术有机融合，本项目为智慧社区建设提供了一个可落地、可扩展的技术框架，具有重要的实践应用价值和推广前景。

4. 测试与成果分析

4.1 功能测试

小车按既定路线行驶、采集图片、是否社区人员的识别、车牌号的识别等功能均按预期正确实现。

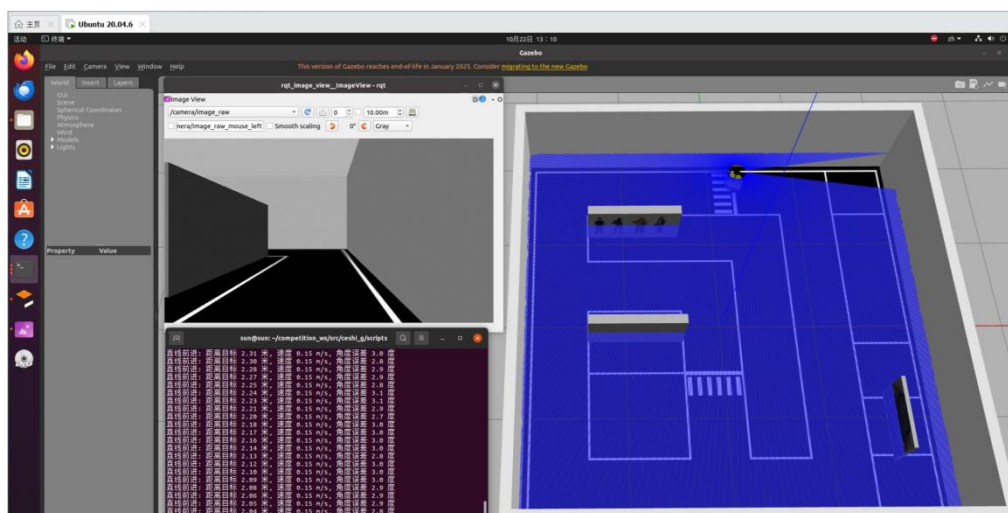


图4-1 小车正常行驶示例1

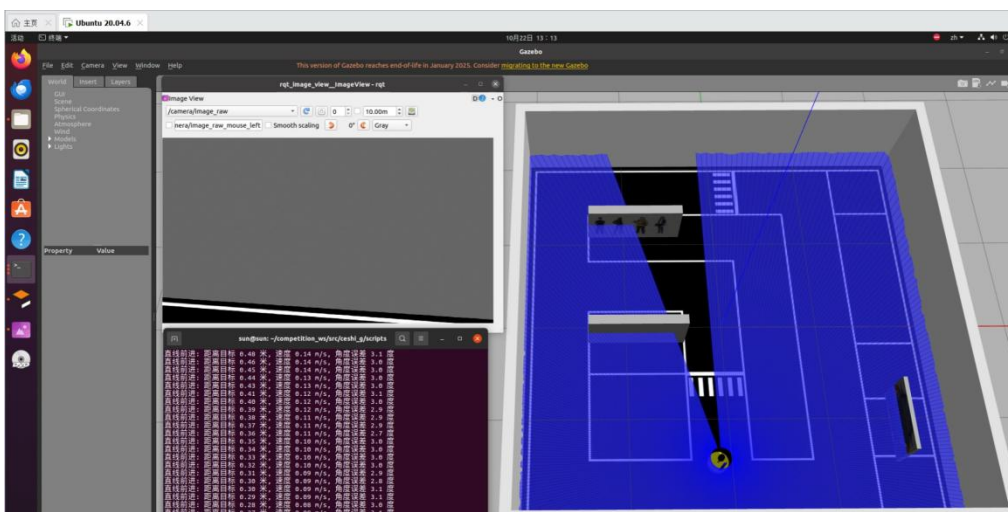


图4-2 小车正常行驶示例2

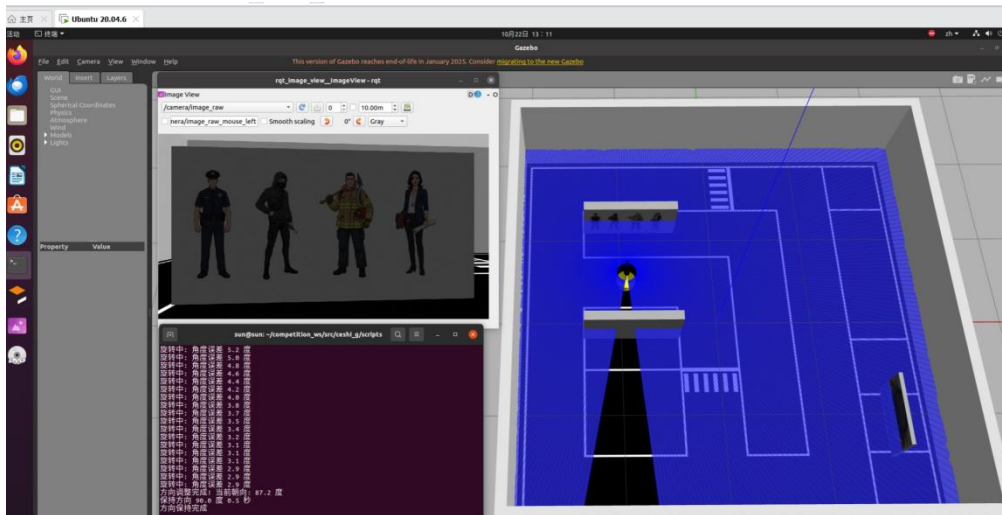


图4-3 小车采集图片示例1

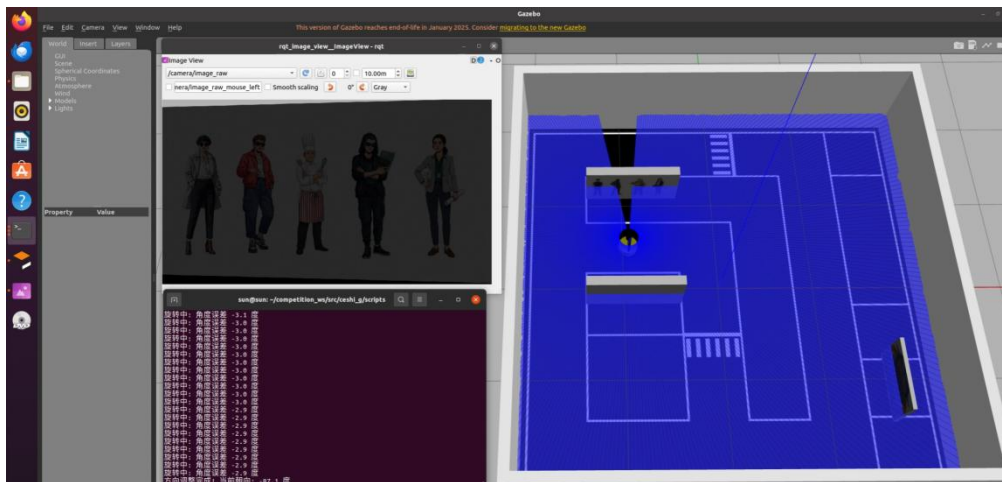


图4-4 小车采集图片示例2

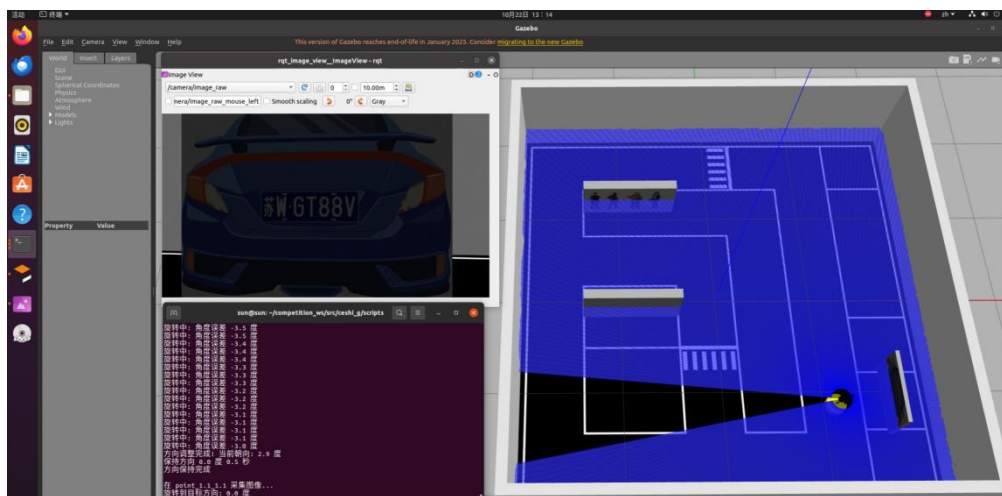


图4-5 小车采集图片示例3



图4-6 原始图像1



图4-7 识别结果示例1



图4-8 原始图像2



图4-9 识别结果示例2



图4-10 原始图像3



图4-11 原始图像3

```
(1, 3, 384, 640)
人物识别结果: 位置: point 0.9 0.5, 方向: 90deg
总人数: 4, 社区人员: 3, 非社区人员: 1
人物识别结果已保存: /home/sun/competition_ws/src/ceshi_g/images/results/people_r
result_point_0.9_0.5_90deg_4608.93.jpg
采集和识别完成: capture_point_0.9_0.5_90deg_4608.83.jpg
采集位置: (-0.94, 0.49), 朝向: 87.2°
识别结果: 位置: point_0.9 0.5, 方向: 90deg
总人数: 4, 社区人员: 3, 非社区人员: 1
旋转目标方向: 00.0度
```

图4-12 人物识别情况的终端输出1

```
(1, 3, 384, 640)
人物识别结果: 位置: point 0.9 0.5, 方向: -90deg
总人数: 5, 社区人员: 4, 非社区人员: 1
人物识别结果已保存: /home/sun/competition_ws/src/ceshi_g/images/results/people_r
result_point_0.9_0.5_-90deg_4630.40.jpg
采集和识别完成: capture_point_0.9_0.5_-90deg_4630.33.jpg
采集位置: (-0.94, 0.49), 朝向: -87.2°
识别结果: 位置: point_0.9 0.5, 方向: -90deg
总人数: 5, 社区人员: 4, 非社区人员: 1
```

图4-13 人物识别情况的终端输出2

```
原始识别结果: 苏W·GT88V
车牌识别结果: 位置: point_1.1_1.1, 方向: 0deg
车牌号码: 苏WGT88V
车牌识别结果已保存: /home/sun/competition_ws/src/ceshi_g/images/results/car_resu
lt_point_1.1_1.1_0deg_4742.32.jpg
采集和识别完成: capture_point_1.1_1.1_0deg_4742.20.jpg
采集位置: (1.11, -1.15), 朝向: 2.8°
识别结果: 位置: point_1.1_1.1, 方向: 0deg
车牌号码: 苏WGT88V
```

图4-14 车牌号识别情况的终端输出

4.2 性能测试

对图像采集时小车的拍摄角度以及图像增强时的参数 α 进行适当变换,发现均可得到正确的结果。测试结果篇幅较长,详见附录3。

4.3 成果展示

00: 00, 开始进行gazebo仿真地图的搭建。

00: 36, 插入指定赛道。

00: 43, 插入赛道周围的围墙。

00: 51, 在模型中插入人群与车牌立牌。

01: 20, 将创建好的完整赛道模型保存。

01: 51, 集成赛道与小车。

02: 17, 小车摄像头高度修改说明。

02: 59, 小车开始自主运动。

03: 41, 到达人群区域, 进行北侧图像采集, 并展示识别结果。

03: 57, 进行南侧图像采集, 并展示识别结果。

04: 29, 到达车牌区域、开始进行图像采集。

05: 02, 小车回到初始位置, 小车停止运动。

05: 03, 相关代码思路说明。

5. 总结与展望

5.1 工作总结

本项目围绕智慧社区治理的核心需求，成功设计并实现了一套集成环境感知、自主导航与智能识别于一体的安防机器人仿真系统。我们所完成的主要工作与取得的成果为：

1. 仿真环境搭建：基于Gazebo平台，成功构建了包含自定义地面纹理、围墙模型及多个任务标识立牌的社区仿真场景（`runway3.world`），并利用XACRO宏语言创建了配备差速驱动、摄像头与激光雷达的机器人模型，为算法验证提供了可靠的虚拟实验平台。

2. 自主导航与图像采集：开发了基于ROS的多目标点路径跟踪系统。通过实时订阅里程计信息，采用基于位置伺服的双层控制策略，实现了机器人在9个预设路径点间的精确移动与稳定停靠，并能在关键位置自动触发图像采集，按场景分类存储。

3. 多模态视觉识别：

人员身份识别：基于YOLOv8框架训练了人员识别模型，能够有效区分社区人员与非社区人员，并结合图像对比度增强技术，提升了在光照不佳条件下的检测准确率。

车牌号识别：采用PP-OCRv3端到端识别框架，通过图像增强、坐标定位、透视变换校正等一系列处理，实现了复杂背景下中英文混合车牌号的准确识别。

4. 系统集成与测试：将导航、采集与识别模块整合于统一的ROS功能包中，通过节点话题通信实现数据流协同。系统功能测试表明，机器人能够完整执行从自主巡检、图像采集到智能识别的全流程任务，系统运行稳定，各项功能均达到预期目标。

5.2 未来展望

尽管本项目在智慧社区的典型场景中实现了机器人自主导航、多目标视觉识别与结构化数据采集等核心功能，初步验证了人工智能技术在社区治理中的可行性与有效性，但作为一个面向实际应用的原型系统，仍存在一定的优化与拓展空间。未来，我们将从以下几个方面持续推进系统的完善与落地应用：

1. 多模态感知融合与动态环境适应能力的提升

当前系统主要依赖视觉传感器与里程计信息，未来可引入红外、声音、激光雷达等多源传感器，构建更加鲁棒的环境感知体系。同时，系统可进一步引入SLAM技术，实现对动态障碍物的实时避障与路径重规划，适应更加复杂多变的社区环境。

2. 模型轻量化与边缘计算部署

目前视觉识别模型（如YOLOv8、PP-OCRv3）对计算资源仍有一定要求。未来可探索模型剪枝、量化、知识蒸馏等轻量化技术，并尝试在Jetson Nano等嵌入式设备上部署，实现真正意义上的“端侧智能”，降低系统成本与能耗。

3. 跨场景通用性与可扩展性增强

本项目聚焦于“人员身份识别”与“车牌识别”两个典型任务，未来可将系统扩展至更多社区治理场景，如垃圾堆放识别、消防通道占用检测、老人行为异常监测等，构建一个模块化、可插拔的“社区AI中台”，支持多种任务的快速接入与灵活调度。

4. 人机协同与交互体验优化

系统未来可集成语音交互、移动端App控制、Web管理后台等功能，支持社区居民与管理人员的双向互动，实现从“无人巡检”到“人机共治”的升级，提升系统的实用性与用户接受度。

5. 跨领域技术迁移与应用拓展

本项目中成熟的机器人导航控制、视觉识别流水线、多模块系统集成等技术，具备向其他垂直领域迁移的潜力。例如，可应用于智慧园区、智慧工厂、智慧农业等场景，实现巡检、监控、识别等任务的自动化与智能化，具备广泛的社会与经济价值。

附录1：参考文献

- [1] [!一命速通!Ubuntu24.04+ROS2+Yolo 检测|保姆级教程 ros2 yolo-CSDN 博客](#)
- [2] [Gazebo 搭建虚拟环境完成 ROS 机器人定位导航仿真及 YOLO 检测识别标记物体 gazebo 仿真识别物体-CSDN 博客](#)
- [3] [【开源数据集】智慧城市之 CCPD 车牌数据集 ccpd 数据集-CSDN 博客](#)
- [4] [【基于 Ubuntu 下 YOLOv5 的目标识别】保姆级教程 | 虚拟机安装 - Ubuntu 安装 - 环境配置 \(Anaconda/Pytorch/Vscode/YOLOv5\) | 全过程图文 by.Akaxi ubuntu yolov5-CSDN 博客](#)
- [5] [PaddleOCR-PP-OCRv4 推理详解及部署实现（上）-CSDN 博客](#)

附录2：核心代码片段

1、车牌号识别

class LicensePlateDetector:

```
    def __init__(self):
```

```
        self.runtime_option = fd.RuntimeOption()
```

```
        self.runtime_option.use_cpu()
```

```
        try:
```

```
            # 车牌 OCR 识别模型
```

```
            self.det_model = fd.vision.ocr.DBDDetector(
```

```
                '/home/sun/competition_ws/src/ceshi_g/model/OCR/ch_PP-OCRv3_det_infer/inference.pdmodel',
```

```
                '/home/sun/competition_ws/src/ceshi_g/model/OCR/ch_PP-OCRv3_det_infer/inference.pdiparams',
```

```
                runtime_option=self.runtime_option
```

```
            )
```

```
            self.cls_model = fd.vision.ocr.Classifier(
```

```
                '/home/sun/competition_ws/src/ceshi_g/model/OCR/ch_ppocr_mobile_v2.0_cls_infer/inference.pdmodel',
```

```
                '/home/sun/competition_ws/src/ceshi_g/model/OCR/ch_ppocr_mobile_v2.0_cls_infer/inference.pdiparams',
```

```
                runtime_option=self.runtime_option
```

```
            )
```

```
            self.rec_model = fd.vision.ocr.Recognizer(
```

```
                '/home/sun/competition_ws/src/ceshi_g/model/OCR/ch_PP-OCRv3_rec_infer/inference.pdmodel',
```

```
                '/home/sun/competition_ws/src/ceshi_g/model/OCR/ch_PP-OCRv3_rec_infer/inference.pdiparams',
```

```
                '/home/sun/competition_ws/src/ceshi_g/model/OCR/ppocr_keys_v1.txt',
```

```
                runtime_option=self.runtime_option
```

```
            )
```

```
        except Exception as e:
```

```
            print(f"模型加载失败: {e}")
```

```
            raise
```

```
    # 配置 OCR 参数
```

```
    self.det_model.preprocessor.max_side_len = 960
```

```
    self.det_model.postprocessor.det_db_thresh = 0.3
```

```
    self.det_model.postprocessor.det_db_box_thresh = 0.6
```

```

self.det_model.postprocessor.det_db_unclip_ratio = 1.5
self.cls_model.postprocessor.cls_thresh = 0.9

# 创建 PPOCRv3 流水线
self.ppocr_v3 = fd.vision.ocr.PPOCRv3(
    det_model=self.det_model,
    cls_model=self.cls_model,
    rec_model=self.rec_model
)
self.ppocr_v3.cls_batch_size = 1
self.ppocr_v3.rec_batch_size = 6

def recognize_plate_text(self, plate_img):
    """识别车牌文字"""
    return self.ppocr_v3.predict(plate_img)

def extract_plate_region(self, image, coordinates):
    """根据坐标从图像中提取车牌区域"""
    if image is None:
        raise ValueError("图像为空")

    pts = np.array(coordinates, dtype=np.float32)

    # 计算边界矩形
    x, y, w, h = cv2.boundingRect(pts.astype(np.int32))

    # 提取车牌区域
    padding = 5
    x_start = max(0, x - padding)
    y_start = max(0, y - padding)
    x_end = min(image.shape[1], x + w + padding)
    y_end = min(image.shape[0], y + h + padding)

    plate_region = image[y_start:y_end, x_start:x_end]

    return plate_region

def perspective_correction(self, image, coordinates):
    """透视变换校正"""
    src_pts = np.array(coordinates, dtype=np.float32)

    x, y, w, h = cv2.boundingRect(src_pts.astype(np.int32))

    dst_pts = np.array([

```

```

        [0, 0],
        [w, 0],
        [w, h],
        [0, h]
    ], dtype=np.float32)

    matrix = cv2.getPerspectiveTransform(src_pts, dst_pts)
    corrected = cv2.warpPerspective(image, matrix, (w, h))

    return corrected

def preprocess_plate_image(self, plate_img):
    """预处理车牌图像以提高识别准确率"""
    if plate_img is None or plate_img.size == 0:
        return None

    # 调整图像大小，确保足够大以便识别
    height, width = plate_img.shape[:2]
    if width < 100:
        new_width = 200
        new_height = int(height * (200 / width))
        plate_img = cv2.resize(plate_img, (new_width, new_height))

    # 增强对比度 - 使用 LAB 色彩空间
    lab = cv2.cvtColor(plate_img, cv2.COLOR_BGR2LAB)
    lab[:, :, 0] = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(lab[:, :, 0])
    enhanced = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)

    # 轻微锐化
    kernel = np.array([[ -1, -1, -1], [-1, 9, -1], [-1, -1, -1]])
    sharpened = cv2.filter2D(enhanced, -1, kernel)

    return sharpened

def clean_plate_text(self, text):
    """清理和验证车牌号码"""
    if not text:
        return "未识别到车牌"

    import re
    cleaned = re.sub(r'^\w\u4e00-\u9fff', "", text)

    allowed_chars = '0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ 京沪津渝冀晋'
    蒙辽吉黑苏浙皖闽赣鲁豫鄂湘粤桂琼川贵云藏陕甘青宁新'

```

```

cleaned = ".join(char for char in cleaned if char in allowed_chars)

if 5 <= len(cleaned) <= 8:
    return cleaned
else:
    return f"识别结果待确认: {cleaned}" if cleaned else "未识别到车牌"

def recognize_plate_from_image(self, image, coordinates):
    """从图像中识别车牌"""
    try:
        print("正在提取车牌区域...")

        # 直接从图像中提取车牌区域
        plate_img = self.extract_plate_region(image, coordinates)
        corrected_plate = self.perspective_correction(image, coordinates)

        # 检查提取的车牌区域是否有效
        if corrected_plate.size == 0:
            return "无法提取有效车牌区域", None

        print("正在进行车牌识别...")

        # 预处理车牌图像
        processed_plate = self.preprocess_plate_image(corrected_plate)

        if processed_plate is None:
            return "车牌图像预处理失败", None

        # OCR 识别
        ocr_result = self.recognize_plate_text(processed_plate)

        # 处理识别结果
        recognized_text = ""
        if ocr_result.text:
            recognized_text = ".join(ocr_result.text)
            print(f"原始识别结果: {recognized_text}")

        cleaned_text = self.clean_plate_text(recognized_text)

        return cleaned_text

    except Exception as e:
        return f"识别过程中出现错误: {str(e)}"

```

```

def visualize_result(self, enhanced_img, plate_img, coordinates, plate_number):
    """可视化识别结果"""
    # 在增强图像上绘制车牌区域
    result_img = enhanced_img.copy()
    pts = np.array(coordinates, dtype=np.int32)
    cv2.polylines(result_img, [pts], True, (0, 255, 0), 2)

    # 将 OpenCV 图像转换为 PIL 图像以支持中文
    result_img_rgb = cv2.cvtColor(result_img, cv2.COLOR_BGR2RGB)
    pil_img = Image.fromarray(result_img_rgb)
    draw = ImageDraw.Draw(pil_img)

    # 字体路径
    font_paths = [
        "/home/sun/competition_ws/src/ceshi_g/ziti/msyh.ttc"
    ]

    font = None
    for font_path in font_paths:
        if os.path.exists(font_path):
            try:
                font = ImageFont.truetype(font_path, 20)
                break
            except Exception:
                continue

    # 计算文本位置 - 在车牌框上方显示
    x_coors = [pt[0] for pt in coordinates]
    y_coors = [pt[1] for pt in coordinates]
    text_x = min(x_coors)
    text_y = min(y_coors) - 30 # 在车牌框上方留出空间

    # 确保文本位置在图像范围内
    if text_y < 10:
        text_y = max(y_coors) + 10

    if font is None:
        # 使用 OpenCV 显示英文
        display_text = f"Plate: {plate_number}"
        cv2.putText(result_img, display_text, (text_x, text_y),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
        return result_img

    # 添加识别结果文本

```

```

text = f"车牌: {plate_number}"
draw.text((text_x, text_y), text, font=font, fill=(255, 0, 0))

# 转换回 OpenCV 格式
result_img = cv2.cvtColor(np.array(pil_img), cv2.COLOR_RGB2BGR)

return result_img

```

2、人物识别代码

```

class PeopleDetector:
    def __init__(self):
        # 加载训练好的模型
        self.model = YOLO("/home/sun/competition_ws/src/ceshi_g/model/people/best.pt")

    def detect_people(self, image):
        """检测图像中的人物并返回检测结果和可视化图像"""
        try:
            # 执行检测
            results = self.model(image)

            total = 0
            community = 0 # 社区人员（类别 0）
            non_community = 0 # 非社区人员（类别 1）

            # 获取可视化结果
            result_img = None
            for result in results:
                # 获取可视化图像（带检测框）
                result_img = result.plot()

                # 获取类别名称映射表和检测到的类别
                class_names = result.names
                detected_classes = [class_names[int(cls_id)] for cls_id in result.bboxes.cls]

            # 统计当前图片的结果
            total = len(detected_classes)

            # 遍历每个检测目标
            for i in range(total):
                cls_id = int(result.bboxes.cls[i])
                if cls_id == 0:
                    community += 1
                elif cls_id == 1:

```

```

        non_community += 1

    return total, community, non_community, result_img

except Exception as e:
    print(f'人物检测失败: {e}')
    return 0, 0, 0, image

def visualize_people_result(self, image, total, community, non_community):
    """在图像上添加人物识别统计信息"""
    try:
        result_img = image.copy()

        # 使用 PIL 绘制中文文本
        result_img_rgb = cv2.cvtColor(result_img, cv2.COLOR_BGR2RGB)
        pil_img = PILImage.fromarray(result_img_rgb)
        draw = ImageDraw.Draw(pil_img)

        # 加载中文字体
        font_path = "/home/sun/competition_ws/src/ceshi_g/ziti/msyh.ttc"
        if os.path.exists(font_path):
            try:
                font = ImageFont.truetype(font_path, 20)
                stats_text = f'总人数: {total}, 社区人员: {community}, 非社区人员: {non_community}'

                draw.text((10, 10), stats_text, font=font, fill=(0, 255, 0))
            except Exception as e:
                print(f'加载中文字体失败, 使用默认字体: {e}')
                stats_text = f'Total: {total}, Community: {community}, Non-community: {non_community}'

                draw.text((10, 10), stats_text, fill=(0, 255, 0))
        else:
            stats_text = f'Total: {total}, Community: {community}, Non-community: {non_community}'

            draw.text((10, 10), stats_text, fill=(0, 255, 0))

        # 转换回 OpenCV 格式
        result_img = cv2.cvtColor(np.array(pil_img), cv2.COLOR_RGB2BGR)

    return result_img

except Exception as e:
    print(f'可视化人物结果时出错: {e}')
    return image

```


3、小车运动与图像采集代码

class RobotController:

def __init__(self):

```
    rospy.init_node('robot_controller', anonymous=True)
    self.pub = rospy.Publisher('/cmd_vel', Twist, queue_size=10)
    self.odom_sub = rospy.Subscriber('/odom', Odometry, self.odom_callback)
    self.image_sub = rospy.Subscriber('/camera/image_raw', Image, self.image_callback)
    self.rate = rospy.Rate(10) # 10Hz
```

设置图像保存路径

```
self.home_dir = os.path.expanduser("~")
self.people_dir = os.path.join(self.home_dir,
```

"competition_ws/src/ceshi_g/images/people")

```
self.car_dir = os.path.join(self.home_dir, "competition_ws/src/ceshi_g/images/car")
```

```
self.results_dir = os.path.join(self.home_dir,
```

"competition_ws/src/ceshi_g/images/results")

创建保存目录（如果不存在）

```
os.makedirs(self.people_dir, exist_ok=True)
```

```
os.makedirs(self.car_dir, exist_ok=True)
```

```
os.makedirs(self.results_dir, exist_ok=True)
```

```
print(f'人物图像将保存到: {self.people_dir}')
```

```
print(f'车辆图像将保存到: {self.car_dir}')
```

```
print(f'识别结果将保存到: {self.results_dir}')
```

初始化识别器

```
self.plate_detector = LicensePlateDetector()
```

```
self.people_detector = PeopleDetector()
```

车牌坐标（基于原始图像尺寸）

```
self.plate_coordinates = [(388, 309), (762, 305), (764, 411), (390, 415)]
```

```
self.current_pose = None
```

```
self.current_image = None
```

```
self.cmd = Twist()
```

```
self.image_count = 0 # 用于计数采集的图像
```

def odom_callback(self, msg):

```
    """获取当前位姿"""
```

```
    self.current_pose = msg.pose.pose
```

def image_callback(self, msg):

```

        """获取摄像头图像"""
        self.current_image = msg

def get_current_position(self):
    """获取当前位置和朝向"""
    if self.current_pose is None:
        return 0, 0, 0
    x = self.current_pose.position.x
    y = self.current_pose.position.y

    # 获取偏航角
    orientation = self.current_pose.orientation
    quaternion = (orientation.x, orientation.y, orientation.z, orientation.w)
    euler = tf.transformations.euler_from_quaternion(quaternion)
    yaw = euler[2]

    return x, y, yaw

def wait_for_odom(self):
    """等待接收到里程计数据"""
    while self.current_pose is None and not rospy.is_shutdown():
        rospy.sleep(0.1)
        print("等待里程计数据...")

def wait_for_image(self):
    """等待接收到图像数据"""
    while self.current_image is None and not rospy.is_shutdown():
        rospy.sleep(0.1)
        print("等待图像数据...")

def save_image(self, filename, image_type="people"):
    """保存当前图像为文件到指定目录"""
    if self.current_image is None:
        print("没有可用的图像数据")
        return False, None

    try:
        # 根据图像类型选择保存目录
        if image_type == "people":
            save_dir = self.people_dir
        elif image_type == "car":
            save_dir = self.car_dir
        else:
            print(f'未知的图像类型: {image_type}')

```

```

        return False, None

    # 完整的文件路径
    filepath = os.path.join(save_dir, filename)

    # 将 ROS 图像消息转换为 OpenCV 图像
    cv_image = self.ros_image_to_cv2(self.current_image)
    if cv_image is None:
        return False, None

    # 保存图像
    cv2.imwrite(filepath, cv_image)
    print(f"图像已保存: {filepath}")
    return True, cv_image

except Exception as e:
    print(f"保存图像时出错: {e}")
    return False, None

def ros_image_to_cv2(self, ros_image):
    """将 ROS 图像消息转换为 OpenCV 图像格式"""
    try:
        if ros_image.encoding == 'rgb8':
            # 直接处理 RGB8 格式
            image_data = np.frombuffer(ros_image.data, dtype=np.uint8)
            image_data = image_data.reshape((ros_image.height, ros_image.width, 3))
            return cv2.cvtColor(image_data, cv2.COLOR_RGB2BGR)

        elif ros_image.encoding == 'bgr8':
            # 处理 BGR8 格式
            image_data = np.frombuffer(ros_image.data, dtype=np.uint8)
            image_data = image_data.reshape((ros_image.height, ros_image.width, 3))
            return image_data

        else:
            print(f"不支持的图像格式: {ros_image.encoding}")
            return None

    except Exception as e:
        print(f"转换图像格式时出错: {e}")
        return None

def enhance_image(self, image):
    """对图像进行增强处理"""

```

```

try:
    # 调整图像大小
    scale_factor = 0.5
    image_resized = cv2.resize(image, (0, 0), fx=scale_factor, fy=scale_factor)

    # 应用增强
    alpha = 1.1
    beta = 25
    enhanced_img = cv2.convertScaleAbs(image_resized, alpha=alpha, beta=beta)

    # 调整坐标以适应增强后的图像尺寸
    adjusted_coordinates = []
    for x, y in self.plate_coordinates:
        adjusted_x = int(x * scale_factor)
        adjusted_y = int(y * scale_factor)
        adjusted_coordinates.append((adjusted_x, adjusted_y))

    return enhanced_img, adjusted_coordinates

except Exception as e:
    print(f'图像增强失败: {e}')
    return image, self.plate_coordinates

def recognize_and_save_result(self, image, location_name, direction, image_type):
    """识别图像并保存结果"""
    try:
        # 增强图像
        enhanced_img, adjusted_coors = self.enhance_image(image)

        if image_type == "people":
            # 人物识别
            total, community, non_community, detection_img =
self.people_detector.detect_people(enhanced_img)

            # 生成结果文本
            result_text = f'位置: {location_name}, 方向: {direction}\n 总人数: {total},
社区人员: {community}, 非社区人员: {non_community}'

            print(f'人物识别结果: {result_text}')

            # 在检测图像上添加统计信息
            result_img = self.people_detector.visualize_people_result(detection_img, total,
community,

```

```

non_community)

        # 保存结果图像
        timestamp = rospy.get_time()
        result_filename =
f'people_result_{location_name}_{direction}_{timestamp:.2f}.jpg'
        result_path = os.path.join(self.results_dir, result_filename)
        cv2.imwrite(result_path, result_img)
        print(f'人物识别结果已保存: {result_path}')

        return result_text

    elif image_type == "car":
        # 车牌识别
        plate_number =
self.plate_detector.recognize_plate_from_image(enhanced_img, adjusted_coords)

        # 生成结果文本
        result_text = f'位置: {location_name}, 方向: {direction}\n 车牌号码:
{plate_number}'

        print(f'车牌识别结果: {result_text}')

        # 可视化车牌识别结果
        result_img = self.plate_detector.visualize_plate_result(enhanced_img,
adjusted_coords, plate_number)

        # 保存结果图像
        timestamp = rospy.get_time()
        result_filename =
f'car_result_{location_name}_{direction}_{timestamp:.2f}.jpg'
        result_path = os.path.join(self.results_dir, result_filename)
        cv2.imwrite(result_path, result_img)
        print(f'车牌识别结果已保存: {result_path}')

        return result_text

    except Exception as e:
        print(f'识别过程中出错: {e}')
        return f'识别失败: {e}'

    def capture_and_recognize_at_pose(self, location_name, target_yaw=None, tolerance=0.05,
image_type="people"):

```

```

"""在特定位置和朝向下采集图像并立即识别"""
print(f"\n 在 {location_name} 采集图像并进行识别...")

# 等待图像数据稳定
rospy.sleep(0.5)

# 如果指定了目标朝向，先旋转到该方向
if target_yaw is not None:
    success = self.rotate_to_direction(target_yaw, tolerance=tolerance)
    if not success:
        print(f"无法旋转到目标方向，继续采集图像...")

# 保持稳定一段时间确保图像清晰
rospy.sleep(0.3)

# 生成文件名
timestamp = rospy.get_time()
if target_yaw is not None:
    direction_deg = int(math.degrees(target_yaw))
    filename = f"capture_{location_name}_{direction_deg}deg_{timestamp:.2f}.jpg"
    direction_str = f"{direction_deg}deg"
else:
    filename = f"capture_{location_name}_{timestamp:.2f}.jpg"
    direction_str = "unknown"

# 保存图像并获取图像数据
success, image_data = self.save_image(filename, image_type)

if success and image_data is not None:
    # 立即进行识别
    result = self.recognize_and_save_result(image_data, location_name, direction_str,
image_type)

    current_x, current_y, current_yaw = self.get_current_position()
    print(f"采集和识别完成: {filename}")
    print(f"采集位置: ({current_x:.2f}, {current_y:.2f}), 朝向:
{math.degrees(current_yaw):.1f}°")
    print(f"识别结果: {result}")
    return True
else:
    print(f"图像采集失败")
    return False

```

```

def move_to_point(self, target_x, target_y, linear_speed=0.15, angular_speed=0.3,
tolerance=0.05):
    """移动到指定坐标点 - 完全按照原始代码的控制方式"""
    print(f'从当前位置移动到坐标: ({target_x:.2f}, {target_y:.2f})')

    self.wait_for_odom()

    while not rospy.is_shutdown():
        current_x, current_y, current_yaw = self.get_current_position()

        # 计算目标方向
        dx = target_x - current_x
        dy = target_y - current_y
        distance_to_target = math.sqrt(dx ** 2 + dy ** 2)

        # 如果已经到达目标点
        if distance_to_target < tolerance:
            self.stop_robot()
            print(f'到达目标点! 当前位置: ({current_x:.2f}, {current_y:.2f})')
            return True

        # 计算目标角度
        target_yaw = math.atan2(dy, dx)

        # 角度误差 (规范化到 [-pi, pi])
        angle_error = target_yaw - current_yaw
        if angle_error > math.pi:
            angle_error -= 2 * math.pi
        elif angle_error < -math.pi:
            angle_error += 2 * math.pi

        # 控制逻辑 - 完全按照原始代码
        self.cmd.linear.x = 0.0
        self.cmd.angular.z = 0.0

        # 如果角度误差较大, 先旋转对准目标
        if abs(angle_error) > 0.15: # 约 8.6 度
            # 角度控制
            angular_control = angular_speed * min(1.0, abs(angle_error) / 0.5)
            self.cmd.angular.z = math.copysign(angular_control, angle_error)
            print(f'调整方向: 角度误差 {math.degrees(angle_error):.1f} 度, 距离目标
{distance_to_target:.2f} 米")
        else:
            # 直线前进控制

```

```

        # 接近目标时减速
        if distance_to_target < 0.5:
            speed_factor = distance_to_target / 0.5
        else:
            speed_factor = 1.0

        linear_control = linear_speed * speed_factor
        self.cmd.linear.x = linear_control

        # 小的角度修正, 保持直线运动
        if abs(angle_error) > 0.05: # 约 2.9 度
            self.cmd.angular.z = angle_error * 1.0 # 增加修正增益

        print(f'直线前进: 距离目标 {distance_to_target:.2f} 米, 速度 {linear_control:.2f} m/s, 角度误差 {math.degrees(angle_error):.1f} 度')

        self.pub.publish(self.cmd)
        self.rate.sleep()

    return False

def rotate_to_direction(self, target_yaw, tolerance=0.05, angular_speed=0.3):
    """旋转到指定方向"""
    print(f'旋转到目标方向: {math.degrees(target_yaw):.1f} 度')

    while not rospy.is_shutdown():
        current_x, current_y, current_yaw = self.get_current_position()

        # 角度误差 (规范化到 [-pi, pi])
        angle_error = target_yaw - current_yaw
        if angle_error > math.pi:
            angle_error -= 2 * math.pi
        elif angle_error < -math.pi:
            angle_error += 2 * math.pi

        # 如果已经对准目标方向
        if abs(angle_error) < tolerance:
            self.stop_robot()
            print(f'方向调整完成! 当前朝向: {math.degrees(current_yaw):.1f} 度')
            return True

        # 角度控制
        angular_control = angular_speed * min(1.0, abs(angle_error) / 0.5)
        self.cmd.angular.z = math.copysign(angular_control, angle_error)

```



```

self.cmd.linear.x = 0.0

print(f'旋转中: 角度误差 {math.degrees(angle_error):.1f} 度")

self.pub.publish(self.cmd)
self.rate.sleep()

return False

def maintain_direction(self, target_yaw, duration=1.0, tolerance=0.05):
    """保持指定方向一段时间"""
    print(f'保持方向 {math.degrees(target_yaw):.1f} 度 {duration} 秒")

    start_time = rospy.get_time()
    while rospy.get_time() - start_time < duration and not rospy.is_shutdown():
        current_x, current_y, current_yaw = self.get_current_position()

        # 角度误差 (规范化到 [-pi, pi])
        angle_error = target_yaw - current_yaw
        if angle_error > math.pi:
            angle_error -= 2 * math.pi
        elif angle_error < -math.pi:
            angle_error += 2 * math.pi

        # 角度控制
        if abs(angle_error) > tolerance:
            self.cmd.angular.z = angle_error * 1.0
        else:
            self.cmd.angular.z = 0.0

        self.cmd.linear.x = 0.0

        self.pub.publish(self.cmd)
        self.rate.sleep()

    self.stop_robot()
    print(f'方向保持完成")

def execute_multiple_points_mission(self):
    """执行多点导航并采集图像"""
    try:
        # 等待初始化完成
        rospy.sleep(2)
        self.wait_for_image() # 等待图像数据

```

```

start_x, start_y, start_yaw = self.get_current_position()
print(f'初始位置: ({start_x:.2f}, {start_y:.2f}), 朝向:
{math.degrees(start_yaw):.1f} 度')

# 定义路径点序列
waypoints = [
    (-1.7, 1.7), # 第一个目标点
    (-1.7, 0.5), # 第二个目标点
    (-0.9, 0.5), # 第三个目标点 (需要方向控制和图像采集)
    (-0.1, 0.5), # 第四个目标点
    (-0.1, -1.7), # 第五个目标点
    (1.1, -1.7), # 第六个目标点
    (1.1, -1.1), # 第七个目标点 (需要方向控制和图像采集)
    (1.1, 1.7), # 第八个目标点
    (1.7, 1.7) # 第九个目标点
]

total_distance = 0
previous_x, previous_y = start_x, start_y

for i, (target_x, target_y) in enumerate(waypoints):
    print(f'\n{'=' * 50}')
    print(f'阶段 {i + 1}: 前往目标点 ({target_x:.1f}, {target_y:.1f})')
    print(f'\n{'=' * 50}')

    # 计算阶段距离
    stage_distance = math.sqrt((target_x - previous_x) ** 2 + (target_y -
previous_y) ** 2)
    print(f'本阶段移动距离: {stage_distance:.2f} 米')

    # 执行移动到当前目标点 - 使用原始参数
    success = self.move_to_point(target_x, target_y,
                                linear_speed=0.15, # 使用原始速度
                                angular_speed=0.3,
                                tolerance=0.05)

    if not success:
        print(f'未能到达第 {i + 1} 个目标点')
        break

    # 特殊点方向控制和图像采集
    if i == 2: # (-0.9, 0.5) 点
        print(f'\n在坐标 (-0.9, 0.5) 执行方向控制和图像采集:')

```

```

        # 朝向正北 ( $\pi/2$ ) 并采集图像 - 保存到 people 文件夹并立即识别
        self.rotate_to_direction(math.pi / 2)
        self.maintain_direction(math.pi / 2, duration=0.5)
        self.capture_and_recognize_at_pose("point_0.9_0.5", math.pi / 2,
image_type="people")

        # 朝向正南 ( $-\pi/2$ ) 并采集图像 - 保存到 people 文件夹并立即识别
        self.rotate_to_direction(-math.pi / 2)
        self.maintain_direction(-math.pi / 2, duration=0.5)
        self.capture_and_recognize_at_pose("point_0.9_0.5", -math.pi / 2,
image_type="people")

    elif i == 6: # (1.1, -1.1) 点
        print(f"\n 在坐标 (1.1, -1.1) 执行方向控制和图像采集:")

        # 朝向正东 (0 度) 并采集图像 - 保存到 car 文件夹并立即识别
        self.rotate_to_direction(0.0)
        self.maintain_direction(0.0, duration=0.5)
        self.capture_and_recognize_at_pose("point_1.1_1.1", 0.0,
image_type="car")

        # 更新位置和累计距离
        current_x, current_y, _ = self.get_current_position()
        actual_stage_distance = math.sqrt((current_x - previous_x) ** 2 + (current_y -
previous_y) ** 2)
        total_distance += actual_stage_distance
        previous_x, previous_y = current_x, current_y

        print(f"阶段 {i + 1} 完成! 实际移动距离: {actual_stage_distance:.2f} 米")

    # 到达最后一个目标点后朝向正西
    print(f"\n 到达最后一个目标点, 调整朝向为正西...")
    self.rotate_to_direction(math.pi) # 正西方向是 $\pi$ 弧度

    # 任务总结
    final_x, final_y, final_yaw = self.get_current_position()
    print(f"\n{'=' * 50}")
    print(f"==== 多点导航任务完成 ===")
    print(f"{'=' * 50}")
    print(f"起始位置: ({start_x:.2f}, {start_y:.2f})")
    print(f"最终位置: ({final_x:.2f}, {final_y:.2f})")
    print(f"最终朝向: {math.degrees(final_yaw):.1f} 度")
    print(f"总移动距离: {total_distance:.2f} 米")

```

```

        # 计算最终精度
        final_target_x, final_target_y = waypoints[-1]
        final_error = math.sqrt((final_x - final_target_x) ** 2 + (final_y - final_target_y)
** 2)

        print(f'最终位置误差: {final_error:.3f} 米")

        # 路径总结
        path_str = f'({start_x:.1f}, {start_y:.1f})'
        for point in waypoints:
            path_str += f' → ({point[0]:.1f}, {point[1]:.1f})'
        print(f'\n 路径总结: {path_str}")

        print(f'\n 图像采集和识别任务完成!")

    except rospy.ROSInterruptException:
        self.stop_robot()
    except Exception as e:
        print(f'发生错误: {e}")
        self.stop_robot()

def stop_robot(self, duration=0.5):
    """停止机器人"""
    self.cmd.linear.x = 0.0
    self.cmd.angular.z = 0.0
    start_time = rospy.get_time()
    while rospy.get_time() - start_time < duration and not rospy.is_shutdown():
        self.pub.publish(self.cmd)
        self.rate.sleep()

if __name__ == '__main__':
    try:
        controller = RobotController()
        controller.execute_multiple_points_mission()
    except rospy.ROSInterruptException:
        pass

```

附录3

下方为不同角度情况下，图像增强的线性变化公式中 α 分别为1.0、1.1、1.2、2.0时所得的增强图像的识别结果。

车牌号识别：

角度1：



角度2：



角度3:

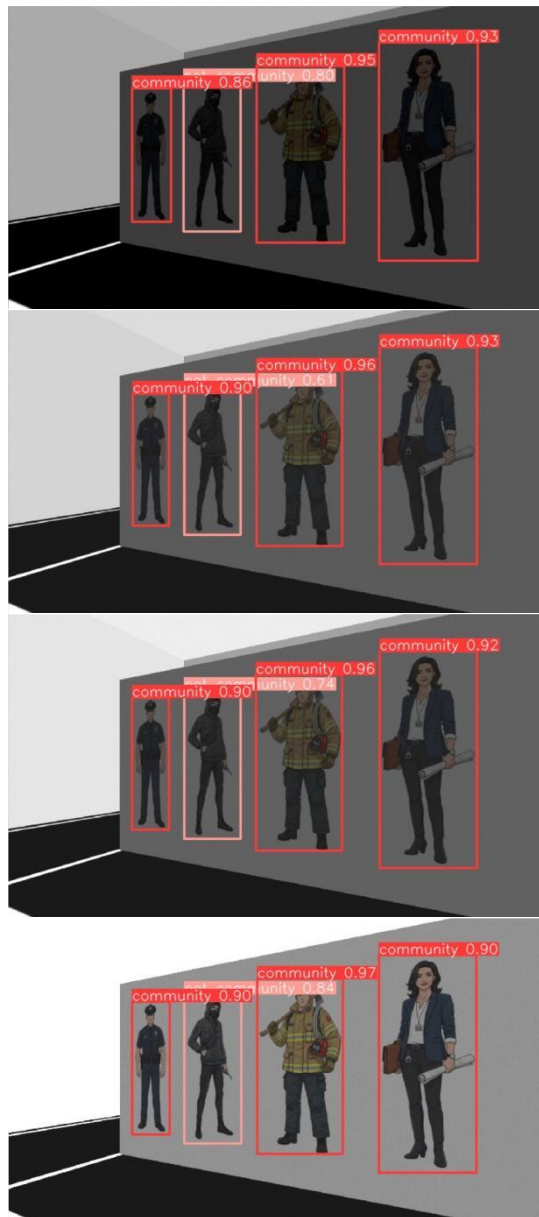




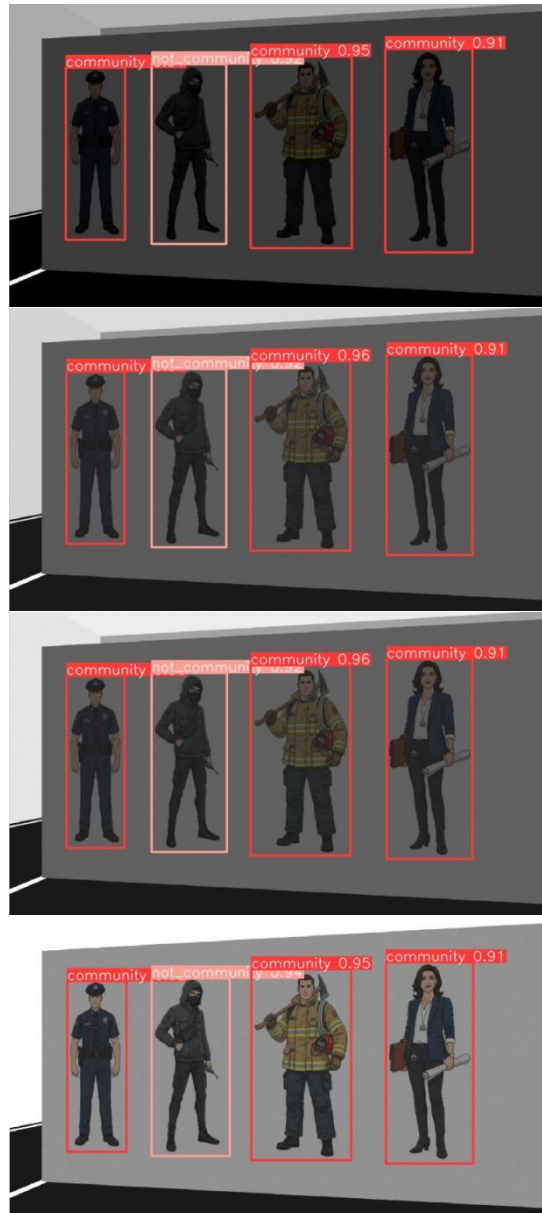
角度4:



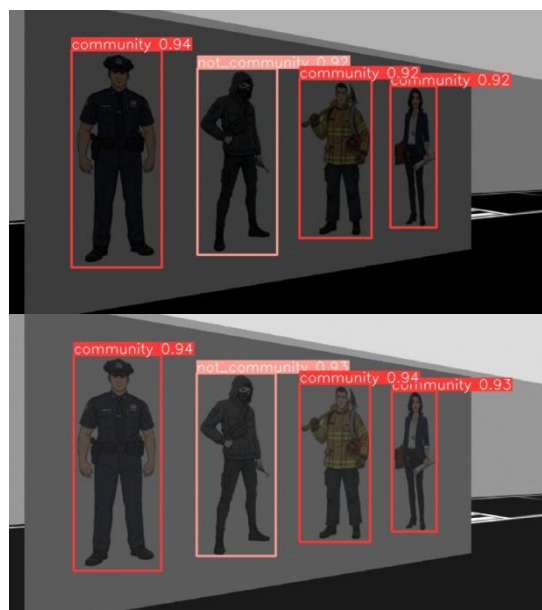
人群1识别：
角度1：

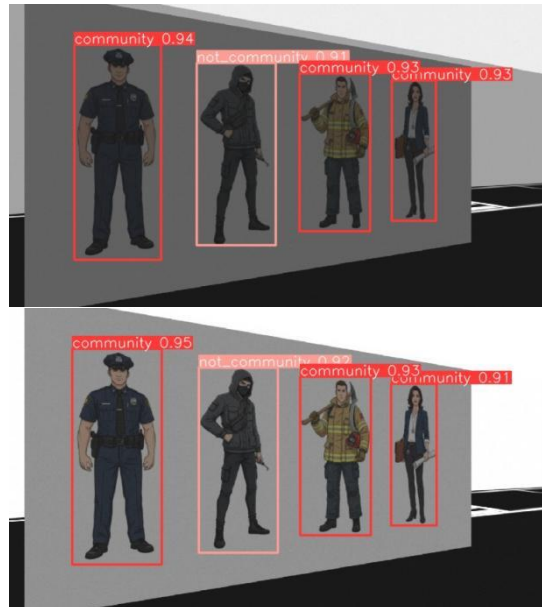


角度2：



角度3:





人群2识别:
角度1:





角度2:



角度3:

